Genetic Algorithm

A complete feature-selection example

Predicting whether a student will pass or fail

Lecture 3 | Classroom worked example

Dr. Fayçal Hamdi

All data are synthetic. This lesson demonstrates the algorithm, not a real predictive claim.

What you will learn

The central question

Which combination of student features should a classifier use to predict **Pass** or **Fail**?

Two different jobs

GA: selects the features.
Classifier: learns to predict using them.

A complete search trace

1. Encode candidates.
2. Train and calculate fitness.
3. Select parents.
4. Crossover and mutation.
5. Replace population members.
6. Apply a stopping condition.
7. Evaluate the final candidate.

Four available student features

Gene	Feature	A value of 1 means	A value of 0 means
A	Attendance	At least 80%	Below 80%
G	Previous grade	At least 70/100	Below 70/100
H	Study hours	At least 3 hours/day	Below 3 hours/day
S	Sleep hours	At least 7 hours/night	Below 7 hours/night

Our prediction target is **Pass (1)** or **Fail (0)**.

A chromosome chooses **which features the model may use**, not which students will pass.

Encoding a candidate solution

Fixed gene order: A G H S

1	0	1	0
Attendance	Grade	Study	Sleep

Chromosome 1010 means use **Attendance** and **Study hours**; ignore the other two features.

The chromosome is neither a prediction nor a score. It is one feature-subset proposal.

How one chromosome receives a fitness score

1. Read chromosome 1010 → keep Attendance + Study

↓

2. Train a decision tree on 14 training students

↓

3. Predict labels for 8 separate validation students

↓

4. Compute accuracy and subtract the feature penalty

Fixed model: decision tree with maximum depth 2. **Fixed split:** 14 train / 8 validation / 4 untouched test.

Fitness and selection criteria

$$F(x) = 100 \frac{\text{correct validation predictions}}{8} - k(x)$$

$k(x)$ is the number of selected features. We maximize $F(x)$.

Candidate with 6/8 correct

Accuracy $= 75\%$, $k = 2$
$F = 75 - 2 = \mathbf{73}$

Candidate with 7/8 correct

Accuracy $= 87.5\%$, $k = 2$
$F = 87.5 - 2 = \mathbf{85.5}$

Higher fitness is preferred; equal accuracy favors fewer features.

Generation 0: The initial population

ID	Chromosome	Selected features	Correct / 8	Accuracy	F
A	0101	Grade + Sleep	6	75.0%	**73.0**
B	1010	Attendance + Study	7	87.5%	**85.5**
C	1001	Attendance + Sleep	4	50.0%	**48.0**
D	1100	Attendance + Grade	5	62.5%	**60.5**

All four fitness values come from **four separately trained decision trees**, evaluated on the **same 8 validation records**.

For this trace we fix the initial population to make every operation reproducible.

Check one fitness calculation: 1010

Attendance + Study hours

ID	Actual	Predicted	Correct?
V1	Pass	Pass	Yes
V2	Fail	Fail	Yes
V3	Fail	Fail	Yes
V4	Pass	Pass	Yes
V5	Fail	Fail	Yes
V6	Pass	Pass	Yes
V7	Fail	Fail	Yes
V8	Fail	Pass	**No**

Correct predictions: 7 of 8

$$\text{Accuracy} = \frac{7}{8} = 87.5\%$$

$$k(1010) = 2$$

$$F(1010) = 87.5 - 2 = 85.5$$

The classifier makes one validation error, on V8.

Parent selection: Tournament of size 2

Selection rule: sample two candidates; the one with the **higher fitness** wins and becomes a parent.

Tournament 1

A: 0101 $F = 73.0$

versus

C: 1001 $F = 48.0$

Winner: Parent 1 = A

Tournament 2

B: 1010 $F = 85.5$

versus

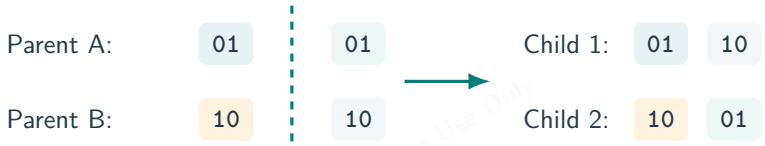
D: 1100 $F = 60.5$

Winner: Parent 2 = B

This example fixes the tournament pairings A–C and B–D. In typical GAs, tournament participants are sampled randomly.

Crossover: Combine the selected parents

One-point crossover after the second gene



Child 1 = 0110 selects Grade + Study hours. This combination was **not present in either selected parent**.

Crossover creates new combinations, but it does not guarantee improvement.

Course Use Only

Evaluate Child 1: 0110

Selected input columns

Previous grade (G) + Study hours (H)

Train a **new** depth-2 decision tree on only these two columns.

Validation outcomes: **8 of 8 correct**.

100%

Validation accuracy

$k(0110) = 2$

$$F = 100 - 2 = \boxed{98}$$

The best observed fitness rises from 85.5 to 98.0.

For Course Use Only

Mutation: Make a small random change

Change one bit of Child 2

$$1001 \longrightarrow 1000$$

Before: 1001
Attendance + Sleep

Accuracy $= 4/8 = 50\%$
Fitness $= 50 - 2 = 48$

After: 1000
Attendance only

Accuracy $= 5/8 = 62.5\%$
Fitness $= 62.5 - 1 = \mathbf{61.5}$

Mutation is random. **It happened to improve this child**, but it can also reduce fitness.

Replacement: Who survives?

Elitist replacement: pool all 4 old candidates and 2 children; **keep the top 4 by fitness**.

Candidate	Chromosome	Fitness	Decision
New Child 1	0110	**98.0**	**Keep**
Old B	1010	85.5	Keep
Old A	0101	73.0	Keep
Mutated Child 2	1000	61.5	Keep
Old D	1100	60.5	Discard
Old C	1001	48.0	Discard

Generation 1 = {0110, 1010, 0101, 1000}.

Stopping condition: Define it before running GA

Stop when either condition holds:

1. **Target quality reached:** a candidate has **100% validation accuracy with at most 2 features**.
2. **Search budget exhausted:** we have completed **3 generations** without reaching the target.

Child 0110 satisfies condition 1

$$\text{Accuracy} = 8/8 = 100\%$$
$$k = 2$$
$$F = 98$$

STOP

Return best found:
0110

A stopping rule is not a mathematical proof of global optimality. It is a rule for ending the computation.

FCIS – For Course Use

What does the selected model predict?

Best found chromosome: `0110` $\Rightarrow$ **Grade + Study hours**

Learned toy decision rule

Predict **Pass** if

Grade is high
AND
Study hours are high.

Otherwise, predict **Fail**.

G	H	Prediction
0	0	Fail
0	1	Fail
1	0	Fail
1	1	**Pass**

G = previous grade high?
H = study hours high?

The synthetic data were constructed to make this rule learnable; it is not an assertion about real students.

Course Use Only

Final assessment: Untouched test students

ID	A	G	H	S	Actual	Predicted
X1	1	1	1	1	Pass	Pass
X2	0	1	0	1	Fail	Fail
X3	1	0	1	0	Fail	Fail
X4	0	1	1	0	Pass	Pass

Test accuracy $= 4/4 = 100\%$ **on this tiny synthetic test set.**

Four test examples are far too few to claim 100% real-world accuracy. The test set was **not used** to choose features or calculate GA fitness.

The entire genetic search in one view

Stage	Decision	Result / reason
Encoding	A, G, H, S; 1 = include	Four-bit feature subsets
Fitness	Accuracy (percentage) − number of features	Maximize score
Initial population	0101, 1010, 1001, 1100	Best = 1010, $F = 85.5$
Select parents	Tournament of two	A beats C; B beats D
Crossover	Cut after second gene	0110 and 1001
Mutation	$1001 \rightarrow 1000$	Flip one bit
Replacement	Keep four best from six	Best = 0110, $F = 98$
Stop	100% validation, $\leq$2 features	Target reached in generation 1
Final result	0110	Grade + Study hours

Best found $\neq$ guaranteed globally optimal

For Course Use Only

Why this is a Genetic Algorithm

Genetic Algorithm

- Creates and stores feature combinations.
- Uses **fitness** to select parents.
- Combines parents via crossover.
- Creates variety via mutation.
- Replaces the population over generations.

Machine-learning classifier

- Receives only selected columns.
- Learns patterns from training labels.
- Predicts Pass / Fail.
- Produces validation predictions.
- Supplies the accuracy used in fitness.

GA chooses the inputs. The model learns from those inputs. Fitness judges the choice.

What if the dataset has a billion records?

Problem: GA evaluates many chromosomes. Full model training for every chromosome may dominate the computation time.

- Start with a **representative subset**.
- Use a cheap **proxy model**.
- Stop weak evaluations early.
- **Cache** repeated chromosomes.
- Evaluate candidates in **parallel**.
- Re-evaluate finalists more carefully.

Approximate fitness can mis-rank candidates. Reserve untouched test data for final assessment, and verify shortlisted candidates with an appropriate validation procedure.

CCIS – For Course Use Only

Quick classroom questions

1. Is chromosome 1010 a prediction that a student will pass?
2. Why is its fitness **85.5**, not **87.5**?
3. How can crossover produce 0110 if neither parent is 0110?
4. Does mutation always improve fitness?
5. Why did the search stop after generation 1?
6. Does 8/8 validation accuracy prove we found the globally best model?

Takeaway: GA finds promising candidate solutions under a chosen fitness function and computational budget—not necessarily a proven global optimum.

For Course Use Only

Appendix A: Complete synthetic training data

Each row is a synthetic student. Features are A, G, H, S; $Y = 1$ (Pass) or 0 (Fail). Repeated binary profiles are distinct student records.

ID	A	G	H	S	Y
T01	1	1	1	1	1
T02	1	1	0	1	0
T03	0	1	0	1	0
T04	0	0	1	0	0
T05	0	1	0	0	0
T06	0	0	0	1	0
T07	0	1	1	0	1

ID	A	G	H	S	Y
T08	0	0	0	1	0
T09	1	1	1	0	1
T10	1	0	1	1	0
T11	1	1	1	0	1
T12	0	1	1	1	1
T13	1	0	0	0	0
T14	0	0	0	0	0

CIS – For Course Use Only

Appendix B: Validation set and predictions

ID	A	G	H	S	Actual Y	1010	0110
V1	1	1	1	0	1	1	1
V2	1	1	0	0	0	0	0
V3	1	0	0	1	0	0	0
V4	0	1	1	0	1	1	1
V5	0	0	0	0	0	0	0
V6	0	1	1	1	1	1	1
V7	1	0	0	0	0	0	0
V8	0	0	1	1	0	**1**	0

1010: 7 correct out of 8 $\Rightarrow F = 85.5$
0110: 8 correct out of 8 $\Rightarrow F = 98.0$

For Course Use Only

Appendix C: Reproducibility code (data)

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
# A, G, H, S, Y: outcome Y is last column
train = np.array([
 [1,1,1,1,1], [1,1,0,1,0], [0,1,0,1,0],
 [0,0,1,0,0], [0,1,0,0,0], [0,0,0,1,0],
 [0,1,1,0,1], [0,0,0,1,0], [1,1,1,0,1],
 [1,0,1,1,0], [1,1,1,0,1], [0,1,1,1,1],
 [1,0,0,0,0], [0,0,0,0,0]
])
val = np.array([
 [1,1,1,0,1], [1,1,0,0,0], [1,0,0,1,0],
 [0,1,1,0,1], [0,0,0,0,0], [0,1,1,1,1],
 [1,0,0,0,0], [0,0,1,1,0]
])
```

Install `scikit-learn` and `numpy` to execute the optional code.

Appendix D: Reproducibility code (fitness)

```
def evaluate(chromosome):
    cols = [i for i, bit in enumerate(chromosome)
            if bit == "1"]
    model = DecisionTreeClassifier(max_depth=2,
                                   random_state=0)
    model.fit(train[:, cols], train[:, 4])
    pred = model.predict(val[:, cols])
    correct = int(np.sum(pred == val[:, 4]))
    accuracy = 100 * correct / len(val)
    return correct, accuracy, accuracy - len(cols)

for x in ["0101", "1010", "1001", "1100",
          "0110", "1000"]:
    print(x, evaluate(x))
```

Note: What the example does not prove

The best found chromosome is not necessarily the global optimum.

The validation and test sets are tiny. This synthetic example was designed to explain how feature selection and model evaluation interact.

In a real application:

- Use a suitable evaluation metric (e.g., F1 for imbalanced classes).
- Tune feature penalty and model complexity.
- Avoid selection leakage between training, validation, and test.
- Re-check the selected model on sufficiently large unseen data.